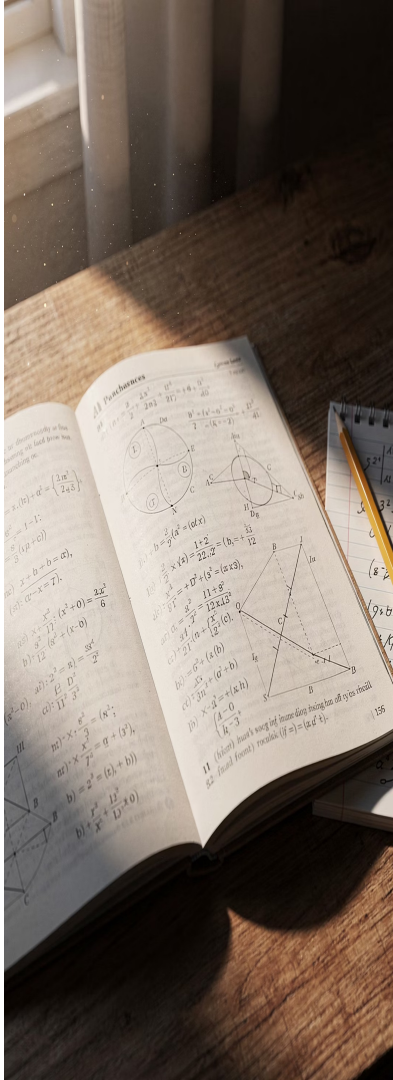
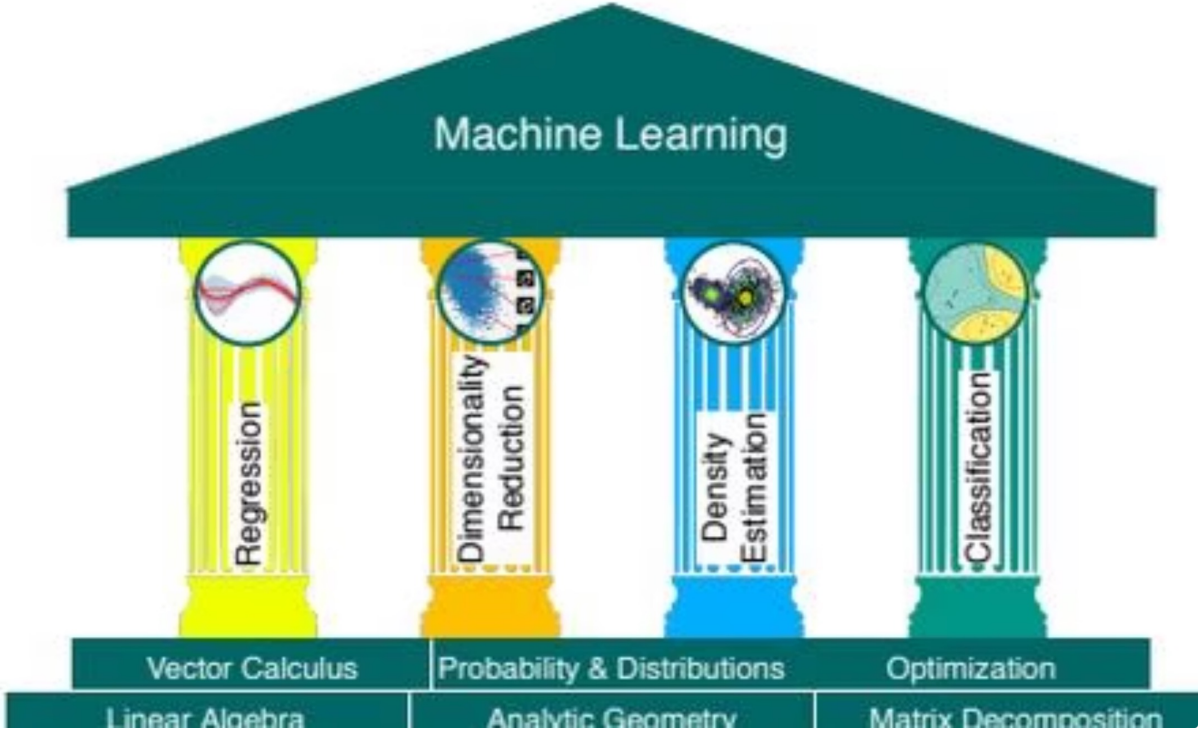
Mathematics for Machine Learning

Pr. Said RAGHAY
Université Cadi Ayyad, Marrakech
s.raghay@uca.ac.ma



Four Pillars, One Foundation



Summary: Mathematics for Machine Learning

These four mathematical pillars — **Linear Algebra**, **Analytic Geometry**, **Matrix Decompositions**, and **Vector Calculus** — provide the rigorous language needed to understand, design, and extend machine learning algorithms.

Linear Algebra

Vectors, matrices, linear mappings, bases, rank, and solving linear systems.

Analytic Geometry

Inner products, norms, angles, orthogonality, projections, and rotations.

Matrix Decompositions

Eigenvalues, Cholesky, eigendecomposition, SVD, and low-rank approximations.

Vector Calculus

Gradients, Jacobians, Hessians, backpropagation, and Taylor approximations.



Chapter 1: Linear Algebra

Linear algebra is the study of vectors and rules to manipulate them. Any object that can be added together and multiplied by a scalar — producing another object of the same kind — qualifies as a vector. This includes geometric vectors, polynomials, audio signals, and tuples in $\mathbb{R}^n$.

What Is a Vector?



Geometric Vectors

Directed line segments with direction and magnitude. Familiar from physics and high school math — they can be added and scaled.



Polynomials

Two polynomials can be added to yield another polynomial, and scaled by a scalar. An unusual but valid instance of a vector.



Elements of $\mathbb{R}^n$

Tuples of n real numbers. Adding two vectors component-wise yields another vector. This is the primary focus of the book.



Audio Signals

Represented as series of numbers. Sums and scalar multiples of audio signals are also audio signals — hence vectors.

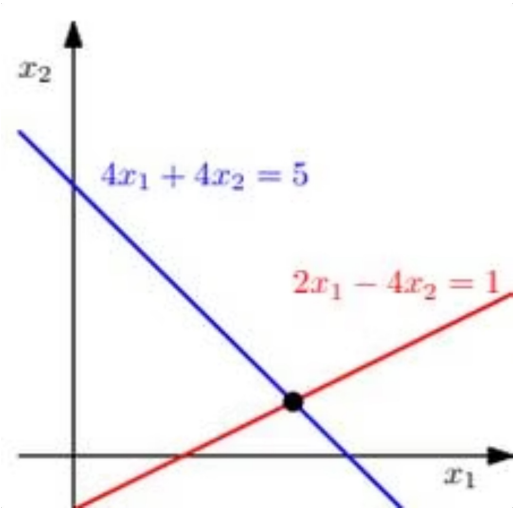
Systems of Linear Equations

General Form

A system of *m* equations in *n* unknowns $x_1, \ldots, x_n$ can be written compactly as : **Ax = b**, where A is the coefficient matrix and b is the right-hand side vector. Solutions may be: **none**, **exactly one**, or **infinitely many**.

Geometric Interpretation

Each linear equation in two variables defines a line. The solution set is the intersection of these lines — which can be a point, a line (same equation), or empty (parallel lines).



Matrices: Definition and Operations

A real-valued **(m, n) matrix A** is an m·n-tuple of elements a_{ij} arranged in m rows and n columns. Matrices compactly represent systems of linear equations and linear mappings.

Addition

Element-wise sum of two matrices of the same size: $(A+B)_{ij} = a_{ij} + b_{ij}$.

Multiplication

$c_{ij} = \Sigma\, a_{il}\, b_{lj}$. Requires neighboring dimensions to match. **Not commutative**: $AB \neq BA$ in general.

Identity Matrix

I_n has 1s on the diagonal and 0s elsewhere. For any A: $I_m A = A I_n = A$.

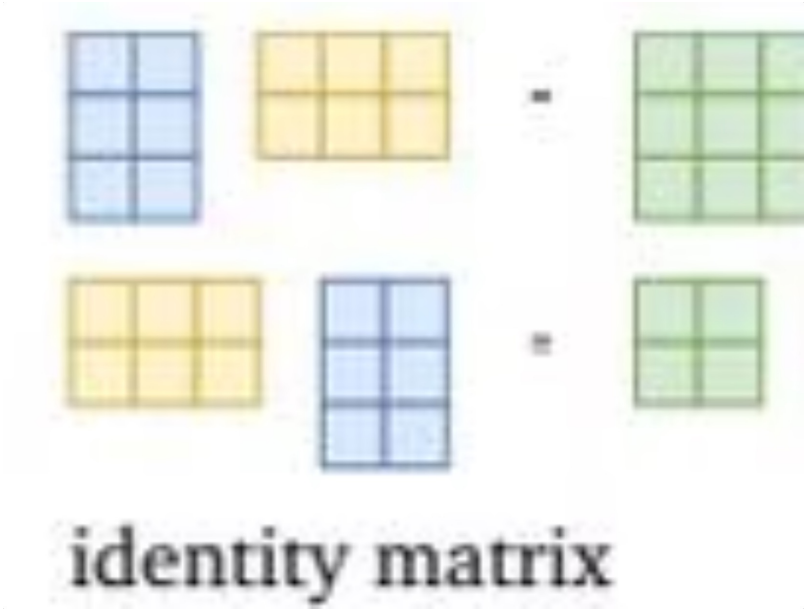
Scalar Multiplication

λA scales each element: $(\lambda A)_{ij} = \lambda a_{ij}$. Satisfies associativity and distributivity.

Inverse and Transpose

Matrix Inverse

For a square matrix $A \in \mathbb{R}^{n \times n}$, if $AB = I = BA$, then $B = A^{-1}$. A matrix with an inverse is called **regular/invertible/nonsingular**; otherwise it is **singular**.
Key properties: $(AB)^{-1} = B^{-1}A^{-1}$ and $(A^{-1})^{\top} = (A^{\top})^{-1}$.

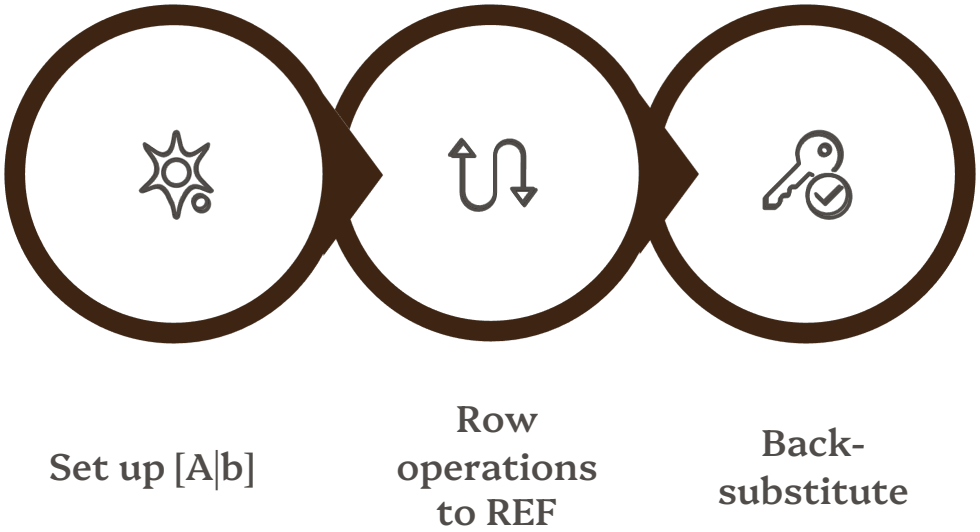


Transpose

For $A \in \mathbb{R}^{m \times n}$, the transpose $A^{\top} \in \mathbb{R}^{n \times m}$ is obtained by writing columns of A as rows.
A matrix is **symmetric** if $A = A^{\top}$.
Key: $(AB)^{\top} = B^{\top}A^{\top}$ and $(A+B)^{\top} = A^{\top} + B^{\top}$.

Solving Linear Systems: Gaussian Elimination

Gaussian elimination transforms any system of linear equations into **row-echelon form (REF)** using three elementary transformations: swapping rows, multiplying a row by a nonzero scalar, and adding rows together.



The general solution combines a **particular solution** (one specific solution) with all solutions to the homogeneous system $Ax = 0$, capturing the full solution space.

Row-Echelon Form and the Minus-1 Trick

Row-Echelon Form (REF)

A matrix is in REF if all-zero rows are at the bottom and each row's leading nonzero entry (pivot) is strictly to the right of the pivot above it. **Basic variables** correspond to pivot columns; **free variables** correspond to non-pivot columns.

The Minus-1 Trick

To read out the kernel of $Ax = 0$ from the reduced REF: augment the matrix to $n \times n$ by inserting rows of the form $[0 \dots -1 \dots 0]$ at positions where diagonal pivots are missing. Columns containing -1 on the diagonal form a basis of the null space.

Vector Spaces and Groups

A **group** $(\mathcal{G}, \otimes)$ satisfies closure, associativity, neutral element, and inverse element. An **Abelian group** additionally satisfies commutativity. A **vector space** $V = (\mathcal{V}, +, \cdot)$ is an Abelian group under addition, with a scalar multiplication satisfying distributivity and associativity.

Closure

$\forall\, x, y \in \mathcal{G} : x \otimes y \in \mathcal{G}$

Neutral Element

$\exists\, e \in \mathcal{G} : x \otimes e = x$

Inverse Element

$\forall\, x \exists\, y : x \otimes y = e$

Associativity

$(x \otimes y) \otimes z = x \otimes (y \otimes z)$

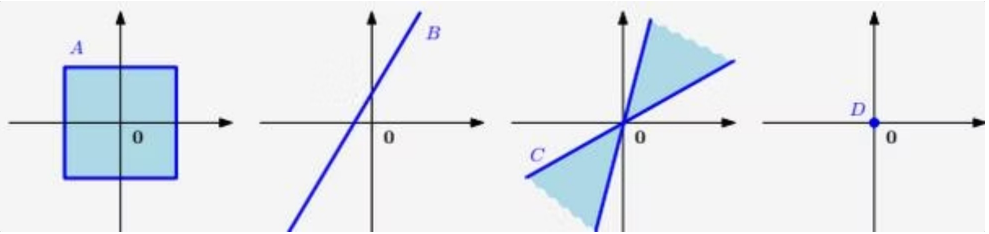
Vector Subspaces

Definition

A subset $U \subseteq V$ is a **vector subspace** if it is non-empty, closed under addition ($x + y \in U$), and closed under scalar multiplication ($\lambda x \in U$). The zero vector must be in U.

Key Examples

- The solution set of $Ax = 0$ is a subspace of $\mathbb{R}^n$
- The solution of $Ax = b$ ($b \neq 0$) is NOT a subspace
- The intersection of any collection of subspaces is itself a subspace
- Every vector space V and $\{0\}$ are trivial subspaces



Linear Independence

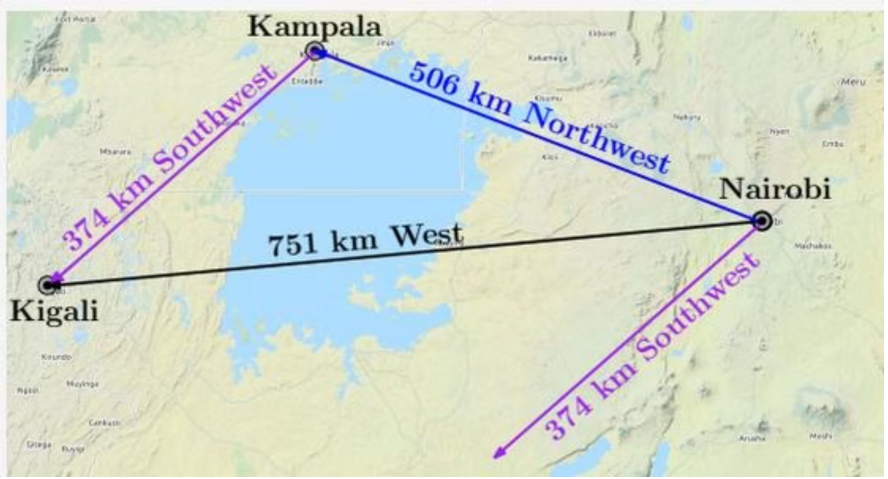
Vectors $x_1, \ldots, x_k$ are **linearly independent** if the only solution to $\lambda_1 x_1 + \ldots + \lambda_k x_k = 0$ is $\lambda_1 = \ldots = \lambda_k = 0$. Otherwise they are **linearly dependent** — at least one vector is redundant.

Practical Check via Gaussian Elimination

Write vectors as columns of a matrix. Apply row operations to reach REF. Pivot columns are linearly independent; non-pivot columns are linear combinations of pivot columns to their left.

Key Properties

If any vector is 0, or two vectors are identical, the set is linearly dependent. A set of m vectors from k linearly independent vectors is dependent if m > k.



Basis and Span

Generating Set & Span

A set $\mathcal{A} = \{x_1, \ldots, x_k\} \subseteq V$ is a **generating set** if every $v \in V$ can be expressed as a linear combination of elements of $\mathcal{A}$. The set of all such combinations is the **span** of $\mathcal{A}$.

Basis

A **basis** is a minimal generating set — equivalently, a maximal linearly independent set. Every vector in V has a **unique** representation as a linear combination of basis vectors. All bases of V have the same number of elements.

The **dimension** of a vector space V, written dim(V), is the number of vectors in any basis of V. It represents the number of independent directions in the space.

Rank of a Matrix

The **rank** of a matrix $A \in \mathbb{R}^{m \times n}$, written $\text{rk}(A)$, is the number of linearly independent columns (which equals the number of linearly independent rows).

$\text{rk}(A) = \text{rk}(A^T)$

Column rank always equals row rank.

Invertibility

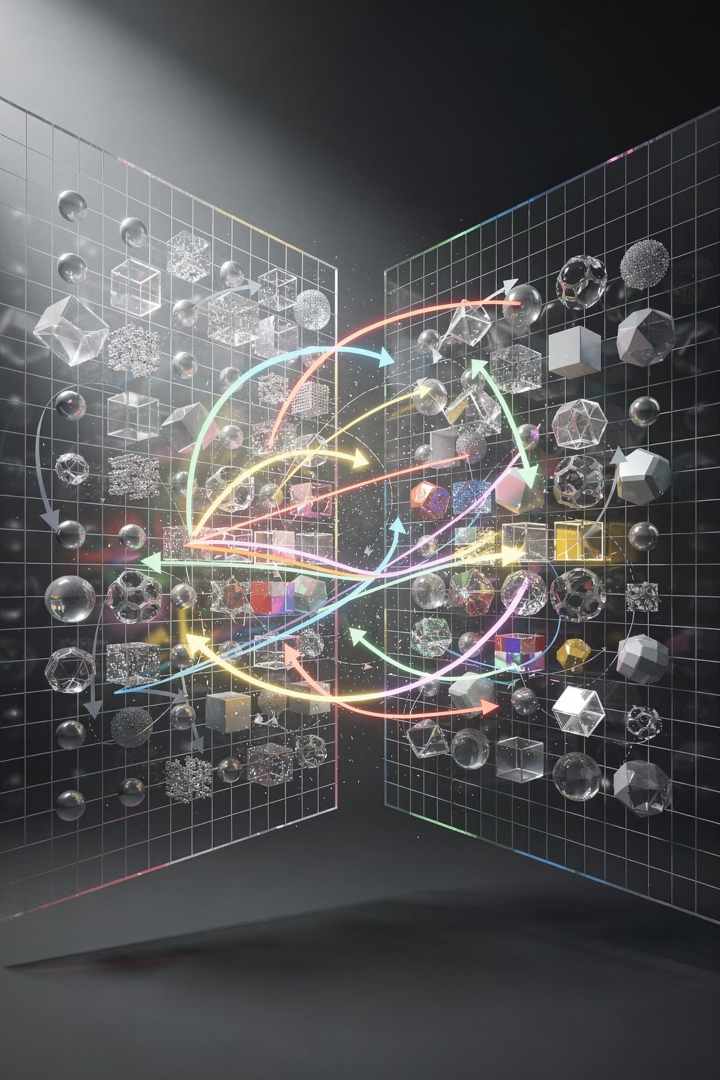
A square $n \times n$ matrix is invertible if and only if $\text{rk}(A) = n$.

Solvability

$Ax = b$ is solvable iff $\text{rk}(A) = \text{rk}([A|b])$.

Null Space Dimension

$\dim(\ker(\Phi)) = n - \text{rk}(A)$ for $A \in \mathbb{R}^{m \times n}$.



Linear Mappings

A mapping $\Phi: V \to W$ is **linear** if for all $x, y \in V$ and $\lambda, \psi \in \mathbb{R}$: $\Phi(\lambda x + \psi y) = \lambda\Phi(x) + \psi\Phi(y)$. Linear mappings preserve the vector space structure — they are also called **vector space homomorphisms** or **linear transformations**.

Isomorphism

Linear and bijective: $\Phi: V \to W$

Endomorphism

Linear mapping to itself: $\Phi: V \to V$

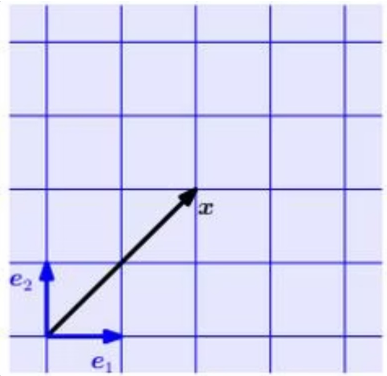
Automorphism

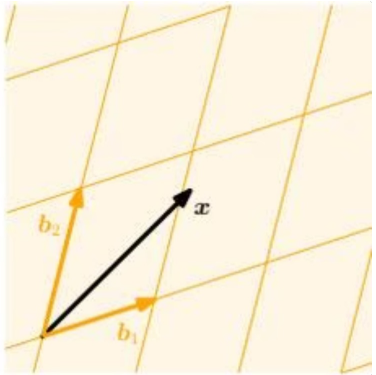
Linear, bijective, to itself: $\Phi: V \to V$

Transformation Matrices & Coordinates

Coordinate Representation

Given an ordered basis $B = (b_1, \ldots, b_n)$ of V, every $x \in V$ has a unique representation $x = \alpha_1 b_1 + \ldots + \alpha_n b_n$. The vector $x = [\alpha_1, \ldots, \alpha_n]^T$ is the **coordinate vector** of x with respect to B.





Transformation Matrix

For a linear mapping $\Phi: V \to W$ with ordered bases B of V and C of W, the **transformation matrix** A_Φ maps coordinate vectors: if $\hat{x}$ are coordinates of x in B, then $\hat{y} = A_\Phi \hat{x}$ gives coordinates of $\Phi(x)$ in C.

Basis Change

When we change the basis in V (from B to B̃) and in W (from C to C̃), the transformation matrix A_Φ is replaced by an **equivalent matrix**:

> $\tilde{A}_\Phi = T^{-1} A_\Phi S$

where S maps coordinates from B̃ to B, and T maps coordinates from C̃ to C. Two matrices A, Ã are **equivalent** if $\tilde{A} = T^{-1}AS$ for regular S, T. They are **similar** if $\tilde{A} = S^{-1}AS$ (same space, endomorphism).

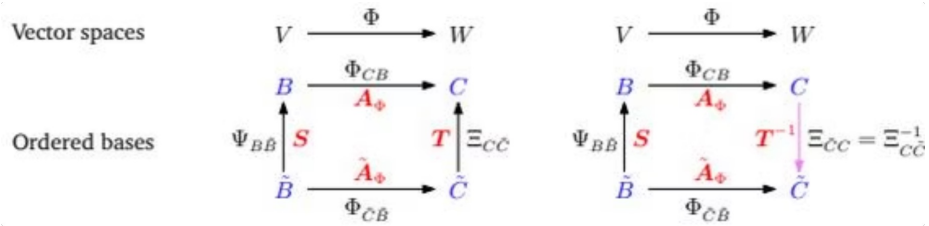


Image and Kernel

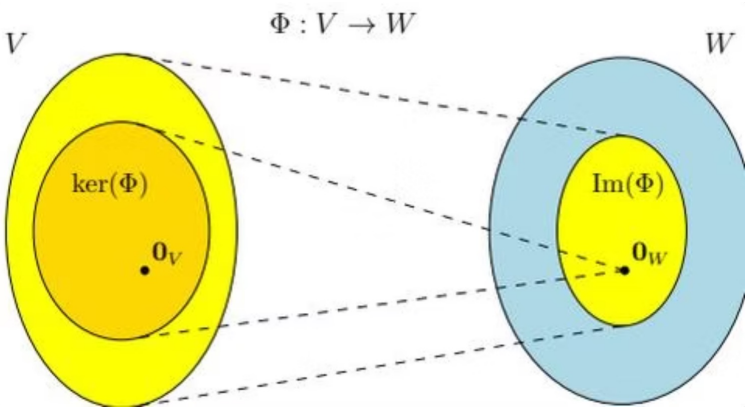
Kernel (Null Space)

$\ker(\Phi) = \{v \in V : \Phi(v) = 0_W\}$ — the set of all vectors mapped to zero. Always a subspace of V. Φ is injective if and only if $\ker(\Phi) = \{0\}$.

Image (Range)

$\mathrm{Im}(\Phi) = \{w \in W \mid \exists\, v \in V : \Phi(v) = w\}$ — the set of all reachable vectors. Always a subspace of W. For a matrix A, $\mathrm{Im}(\Phi)$ = span of columns of A (the **column space**).

Rank-Nullity Theorem: $\dim(\ker(\Phi)) + \dim(\mathrm{Im}(\Phi)) = \dim(V)$. This fundamental result links the kernel and image dimensions to the domain dimension.



Affine Spaces and Mappings

An **affine subspace** $L = x_0 + U$ is a vector subspace U shifted by a support point x_0. Unlike vector subspaces, affine subspaces do not necessarily contain the origin. Examples include lines, planes, and hyperplanes in $\mathbb{R}^n$ that do not pass through the origin.

Lines

1D affine subspaces: $y = x_0 + \lambda x_1$, $\lambda \in \mathbb{R}$. Defined by a support point and a direction vector.

Planes

2D affine subspaces: $y = x_0 + \lambda_1 x_1 + \lambda_2 x_2$. Defined by a support point and two linearly independent direction vectors.

Hyperplanes

$(n-1)$-dimensional affine subspaces of $\mathbb{R}^n$. Solutions to a single linear equation $\lambda_1 x_1 + \ldots + \lambda_n x_n = b$.

Chapter 2: Analytic Geometry

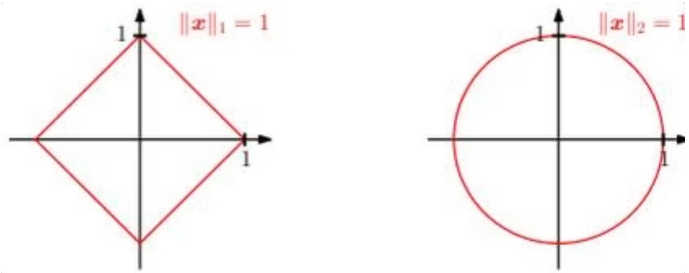
Analytic geometry adds geometric interpretation to linear algebra — computing lengths, distances, and angles between vectors using **inner products**. These tools underpin orthogonal projections, which are central to PCA, regression, and classification.



Norms: Measuring Vector Length

Definition of a Norm

A norm $\|\cdot\|: V \to \mathbb{R}$ assigns each vector a length satisfying: **absolute homogeneity** ($\|\lambda x\| = |\lambda|\|x\|$), **triangle inequality** ($\|x+y\| \le \|x\|+\|y\|$), and **positive definiteness** ($\|x\| \ge 0$, with $\|x\| = 0$ iff $x = 0$).



Common Norms

Manhattan (ℓ_1): $\|x\|_1 = \Sigma|x_i|$ — sum of absolute values. Unit ball is a diamond shape.

Euclidean (ℓ_2): $\|x\|_2 = \sqrt{(\Sigma x_i^2)} = \sqrt{(x^T x)}$ — the standard geometric distance from the origin. Unit ball is a circle/sphere.

Inner Products

An **inner product** $\langle \cdot, \cdot \rangle : V \times V \to \mathbb{R}$ is a positive definite, symmetric bilinear mapping. The standard dot product $x^T y = \Sigma x_i y_i$ is the most familiar example, but inner products can be more general.

Symmetric

$\langle x, y \rangle = \langle y, x \rangle$ for all $x, y \in V$

Positive Definite

$\langle x, x \rangle > 0$ for all $x \neq 0$, and $\langle 0, 0 \rangle = 0$

Bilinear

Linear in each argument separately

Any inner product **induces a norm**: $\|x\| = \sqrt{\langle x, x \rangle}$. However, not every norm comes from an inner product — the Manhattan norm has no corresponding inner product.

Symmetric, Positive Definite Matrices

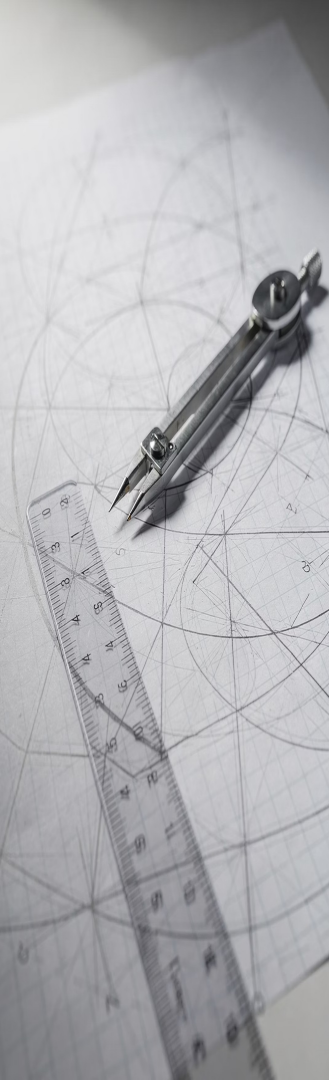
Definition

A symmetric matrix $A \in \mathbb{R}^{nxn}$ is **positive definite** if $x^TAx > 0$ for all $x \neq 0$. It is **positive semidefinite** if $x^TAx \geq 0$.

Every inner product on a finite-dimensional vector space corresponds to a unique symmetric positive definite matrix A via $\langle x, y \rangle = \hat{x^T}A\hat{y}$.

Key Properties

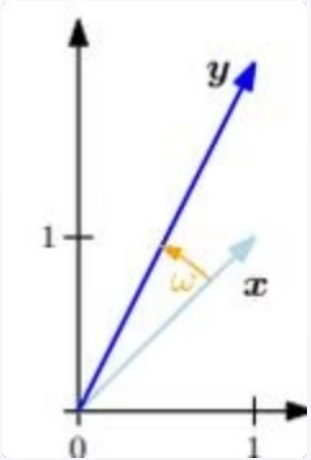
- Null space contains only 0
- All diagonal elements $a_{ii} > 0$
- Defines a valid inner product
- Central to kernel methods (SVMs)
- Used in Cholesky decomposition



Lengths, Distances, and Metrics

The **distance** between vectors x and y in an inner product space is $d(x, y) = \|x - y\| = \sqrt{\langle x-y, x-y \rangle}$. Using the dot product gives the **Euclidean distance**. A metric d satisfies positive definiteness, symmetry, and the triangle inequality.

Inner products and metrics behave in **opposite directions**: very similar vectors yield a *large* inner product but a *small* metric distance.



Angles and Orthogonality

Angle Between Vectors

The angle $\omega \in [0, \pi]$ between nonzero vectors x and y is defined by the Cauchy-Schwarz inequality:

$$\cos \omega = \langle x, y \rangle / (\|x\| \cdot \|y\|)$$

The angle depends on the choice of inner product.

Orthogonality

Vectors x and y are **orthogonal** ($x \perp y$) if $\langle x, y \rangle = 0$. If additionally $\|x\| = \|y\| = 1$, they are **orthonormal**. The zero vector is orthogonal to every vector.
An **orthogonal matrix** A satisfies $AA^T = I = A^TA$, so $A^{-1} = A^T$.

Orthonormal Basis (ONB)

A basis $\{b_1, \ldots, b_n\}$ of V is an **orthonormal basis** if $\langle b_i, b_j \rangle = 0$ for $i \neq j$ (orthogonality) and $\langle b_i, b_i \rangle = 1$ (unit length). The canonical standard basis of $\mathbb{R}^n$ is the most familiar ONB.

01

Start with any basis

Take a set of linearly independent vectors $\{\tilde{b_1}, \ldots, \tilde{b_n}\}$ spanning V.

02

Apply Gram-Schmidt

Iteratively subtract projections to build orthogonal vectors $u_1, \ldots, u_n$.

03

Normalize

Divide each u_k by its norm to obtain unit vectors — the ONB.

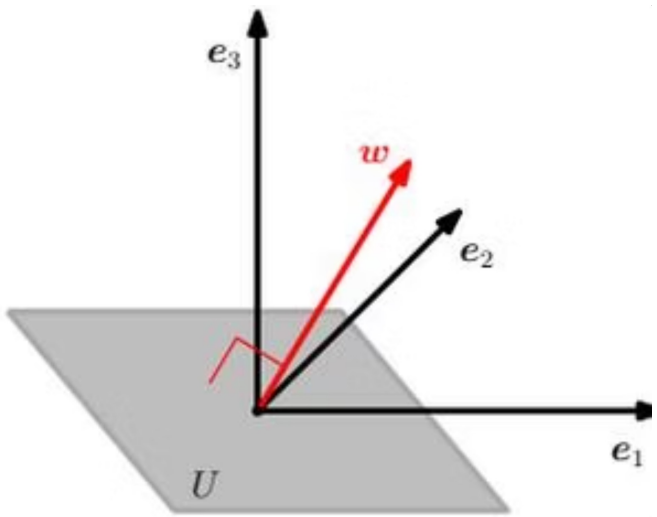
Orthogonal Complement

Definition

For an M-dimensional subspace $U \subseteq V$ (D-dimensional), the **orthogonal complement** $U^\perp$ is the (D−M)-dimensional subspace of all vectors orthogonal to every vector in U. $U \cap U^\perp = \{0\}$.
Any $x \in V$ decomposes uniquely as $x = u + u^\perp$ where $u \in U$ and $u^\perp \in U^\perp$.

Normal Vectors

A plane U in $\mathbb{R}^3$ can be described by its **normal vector** w with $\|w\| = 1$, which spans $U^\perp$. All vectors orthogonal to w lie in U. Orthogonal complements generalize this to hyperplanes in n-dimensional spaces.

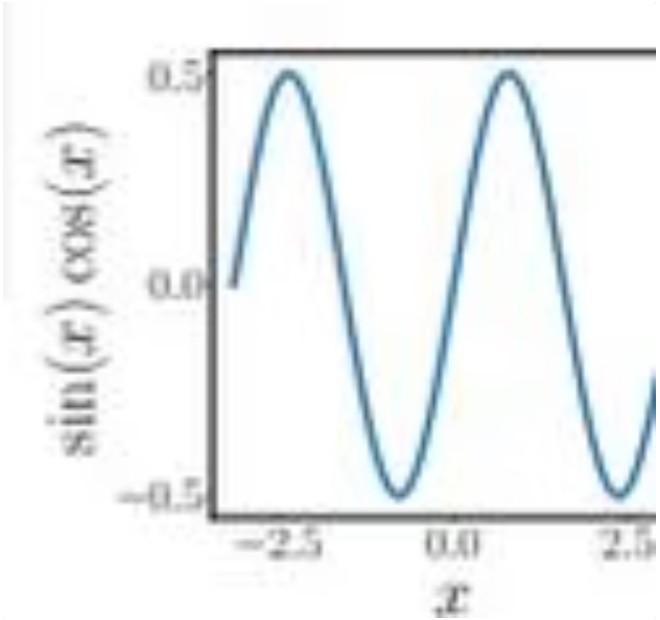


Inner Products of Functions

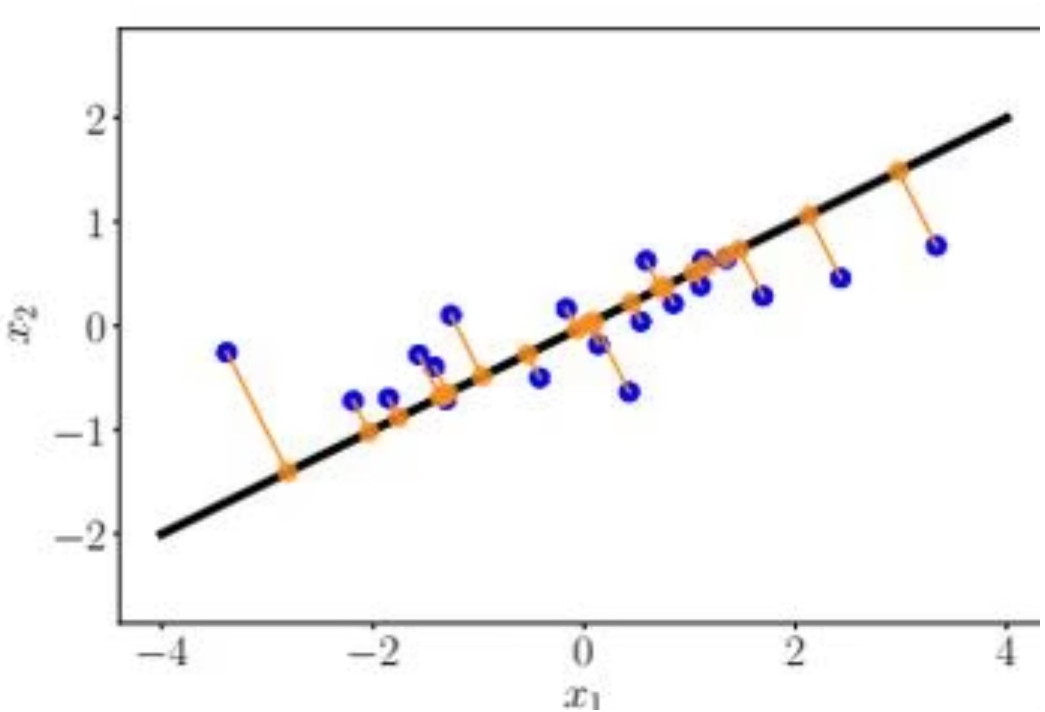
Inner products generalize beyond finite-dimensional vectors. For functions u, v: $\mathbb{R} \to \mathbb{R}$, the inner product is defined as the definite integral:

> $\langle u, v \rangle := \int_a^b u(x)v(x)\,dx$

If this integral equals 0, the functions are **orthogonal**. For example, sin(x) and cos(x) are orthogonal on [−π, π]. The collection {1, cos(x), cos(2x), ...} forms an orthogonal set — the foundation of **Fourier series**.



Orthogonal Projections



A **projection** π: V → U is a linear mapping satisfying $\pi^2 = \pi$. Orthogonal projections minimize the distance between the original vector and its image in the subspace — retaining maximum information. They are fundamental to PCA, regression, and classification.

Projection onto a Line (1D Subspace)

To project $x \in \mathbb{R}^n$ onto a line spanned by $b \in \mathbb{R}^n$, the projection $\pi_U(x) = \lambda b$ must satisfy the orthogonality condition $\langle x - \pi_U(x), b \rangle = 0$.

1 **Find coordinate λ**

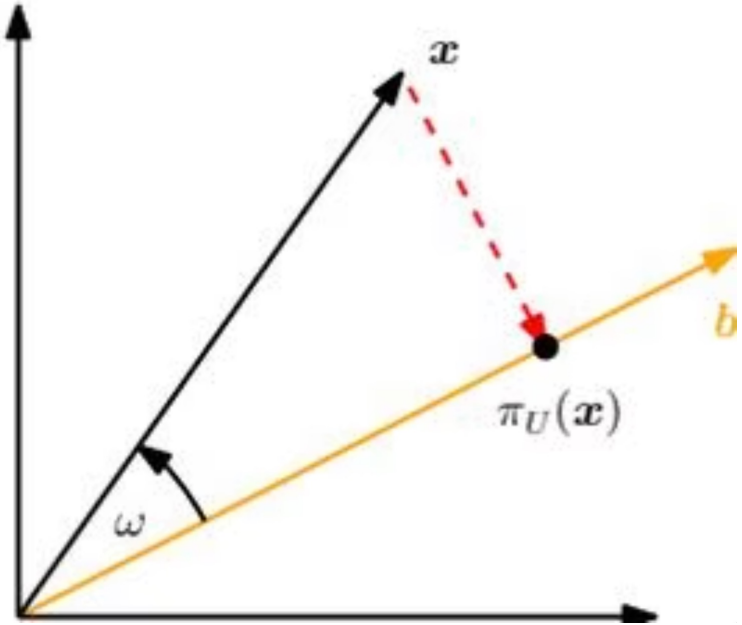
$\lambda = \langle x, b \rangle / \|b\|^2 = b^T x / b^T b$ (using dot product)

2 **Find projection point**

$\pi_U(x) = \lambda b = (b^T x / \|b\|^2)\, b$

3 **Find projection matrix**

$P_\pi = bb^T / \|b\|^2$ — symmetric, rank-1, satisfies $P_\pi^2 = P_\pi$



Projection onto General Subspaces

Normal Equation

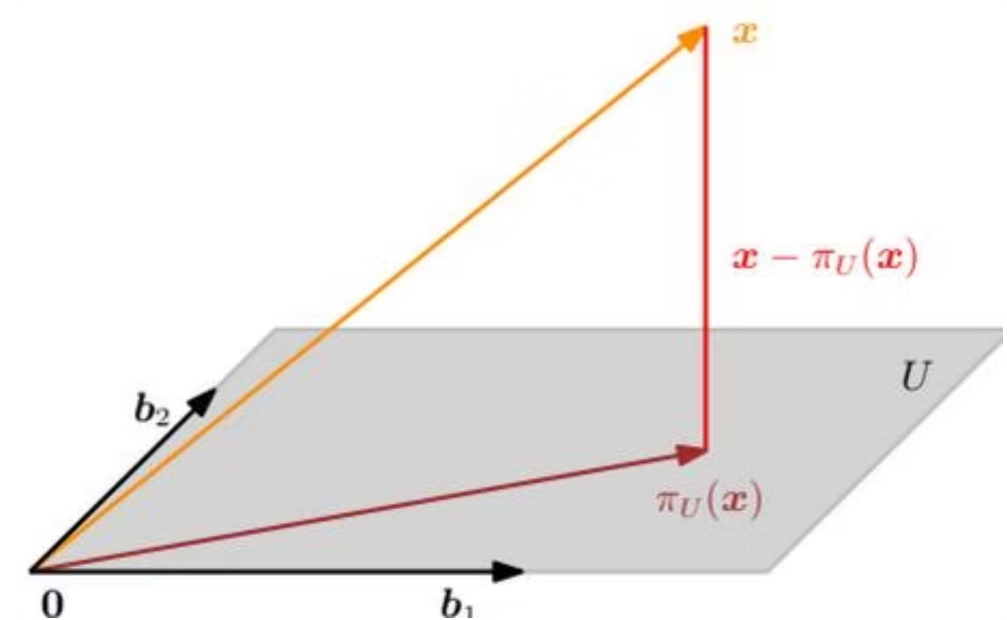
For a subspace U with basis $B = [b_1, \ldots, b_m]$, the projection coordinates λ satisfy the **normal equation**: $B^T B\lambda = B^T x$. The solution is $\lambda = (B^T B)^{-1} B^T x$.

Projection Formula

The projection point: $\pi_U(x) = B(B^T B)^{-1} B^T x$

The projection matrix: $P_\pi = B(B^T B)^{-1} B^T$

If B is an ONB, this simplifies to $\pi_U(x) = BB^T x$ since $B^T B = I$.

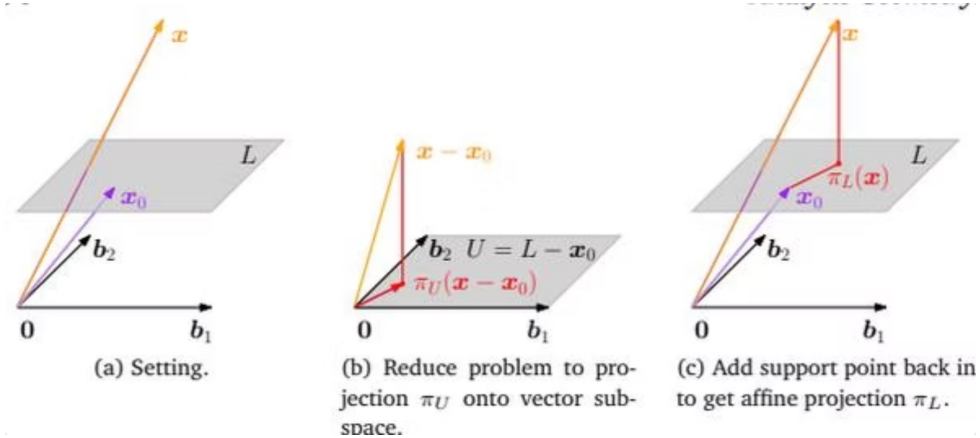


Gram-Schmidt Orthogonalization

Given any basis $(b_1, ..., b_n)$ of V, the **Gram-Schmidt process** constructs an orthogonal basis $(u_1, ..., u_n)$ iteratively:

> $u_1 := b_1 \quad u_k := b_k - \pi_{span[u_1,...,u_{k-1}]}(b_k)$

Each new vector u_k is obtained by subtracting from b_k its projection onto the subspace already spanned. Normalizing the result yields an ONB. This process is foundational for QR decomposition and Krylov subspace methods.



(a) Setting.

(b) Reduce problem to projection π_U onto vector subspace.

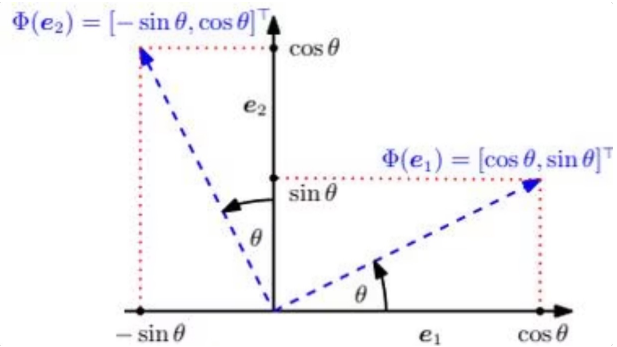
(c) Add support point back in to get affine projection π_L.

Rotations in $\mathbb{R}^2$ and $\mathbb{R}^3$

Rotation in $\mathbb{R}^2$

Rotating the standard basis by angle θ counterclockwise gives the **rotation matrix**:
R(θ) = [[cos θ, −sin θ], [sin θ, cos θ]]

Orthogonal matrices preserve both lengths and angles.



Rotation in $\mathbb{R}^3$

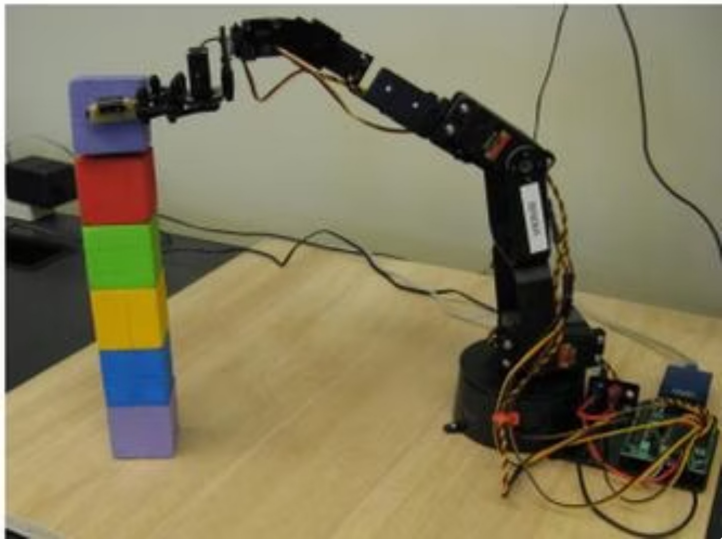
Three elementary rotations about the standard axes e_1, e_2, e_3 (R_1, R_2, R_3). Rotations in 3D are generally **not commutative** — order matters. In 2D only, rotations form an Abelian group.

Givens Rotations in n Dimensions

In n-dimensional Euclidean space, a **Givens rotation** R_{ij}(θ) rotates in the (i, j)-plane while fixing all other dimensions. It is the identity matrix with:

r_{ii} = cos θ Diagonal entry for row i	**r_{ij} = −sin θ** Off-diagonal entry
r_{ji} = sin θ Off-diagonal entry	**r_{jj} = cos θ** Diagonal entry for row j

Rotations preserve distances and angles, and are used extensively in numerical linear algebra (e.g., QR decomposition) and robotics.



Chapter 3: Matrix Decompositions

Matrix decompositions reveal the intrinsic structure of matrices, enabling efficient computation, geometric insight, and powerful machine learning algorithms. Key decompositions include Cholesky, Eigendecomposition, and Singular Value Decomposition (SVD).

Determinant and Trace

Determinant

det(A) is a scalar that encodes the signed volume scaling factor of the linear mapping represented by A. A matrix is invertible if and only if det(A) ≠ 0. For 2×2: det([[a,b],[c,d]]) = ad − bc.

Trace

tr(A) = $\Sigma\, a_{ii}$ — the sum of diagonal elements. Key properties: tr(A+B) = tr(A)+tr(B), tr(λA) = λ tr(A), and tr(AB) = tr(BA) (cyclic property). The trace equals the sum of eigenvalues.

Gaussian elimination is used to compute determinants: det(A) equals the product of pivots (with sign adjustments for row swaps).

Eigenvalues and Eigenvectors

For a square matrix $A \in \mathbb{R}^{n \times n}$, a nonzero vector x is an **eigenvector** with **eigenvalue** λ if $Ax = \lambda x$. Eigenvectors are only scaled — not rotated — by A.

01

Characteristic Polynomial

Solve $\det(A - \lambda I) = 0$ to find eigenvalues $\lambda_1, \ldots, \lambda_n$.

02

Eigenspaces

For each λ, solve $(A - \lambda I)x = 0$ to find the eigenspace $E_\lambda = \ker(A - \lambda I)$.

03

Key Relations

$\operatorname{tr}(A) = \Sigma \lambda_i$ and $\det(A) = \Pi \lambda_i$. Symmetric matrices have real eigenvalues and orthogonal eigenvectors.

Cholesky Decomposition

Definition

Every symmetric, positive definite matrix A can be factored as $A = LL^T$, where L is a lower triangular matrix with positive diagonal entries. L is called the **Cholesky factor** of A.

Applications

- Efficient solution of linear systems $Ax = b$
- Computing matrix determinants: $\det(A) = (\Pi l_{ii})^2$
- Sampling from multivariate Gaussian distributions
- Numerical stability in machine learning algorithms

Eigendecomposition and Diagonalization

A matrix $A \in \mathbb{R}^{nxn}$ with n linearly independent eigenvectors can be **diagonalized**:

> $A = PDP^{-1}$

where $P = [p_1, ..., p_n]$ is the matrix of eigenvectors and $D = \text{diag}(\lambda_1, ..., \lambda_n)$ is the diagonal matrix of eigenvalues. Symmetric matrices are always diagonalizable with an orthogonal P (so $P^{-1} = P^T$), giving $A = PDP^T$.

Diagonalization simplifies matrix powers: $A^k = PD^kP^{-1}$, where $D^k = \text{diag}(\lambda_1^k, ..., \lambda_n^k)$. This is far more efficient than repeated matrix multiplication.

Singular Value Decomposition (SVD)

The **SVD** is the most general matrix decomposition, applicable to any matrix $A \in \mathbb{R}^{m \times n}$:

> $A = U \Sigma V^T$

where $U \in \mathbb{R}^{m \times m}$ and $V \in \mathbb{R}^{n \times n}$ are orthogonal matrices, and $\Sigma \in \mathbb{R}^{m \times n}$ is a diagonal matrix with non-negative **singular values** $\sigma_1 \geq \sigma_2 \geq \ldots \geq 0$ on the diagonal.

SVD: Geometric Interpretation

1

V^T: Rotation/Reflection

Rotates the input space — maps the standard basis to the right singular vectors.

2

Σ: Scaling

Scales along the new axes by the singular values σ_i — stretching or compressing.

3

U: Rotation/Reflection

Rotates the output space — maps to the left singular vectors in the codomain.

The singular values are the square roots of the eigenvalues of A^TA (or AA^T). The columns of V are eigenvectors of A^TA; the columns of U are eigenvectors of AA^T.

Matrix Approximation via SVD

Low-Rank Approximation

The best rank-r approximation of A (in the Frobenius norm sense) is obtained by keeping only the r largest singular values:

$$\hat{A}_r = \sum_{i=1}^{r} \sigma_i u_i v_i^T$$

This is the **Eckart-Young theorem**.

Applications in ML

- Principal Component Analysis (PCA)
- Image compression and denoising
- Latent Semantic Analysis (LSA)
- Recommender systems
- Computing the Moore-Penrose pseudoinverse: $A^+ = V\Sigma^+U^T$

Chapter 4: Vector Calculus

Vector calculus extends differentiation to functions of multiple variables and vector-valued functions. **Gradients** generalize derivatives to higher dimensions and are the engine behind optimization and training of machine learning models.

Differentiation of Univariate Functions

Derivative

The derivative df/dx of $f: \mathbb{R} \to \mathbb{R}$ at x is the limit of the difference quotient. It measures the instantaneous rate of change — the slope of the tangent line.

Key rules: product rule, quotient rule, chain rule. These extend naturally to multivariate settings.

Taylor Series

A smooth function f can be approximated locally by its Taylor polynomial:

$$f(x) \approx \Sigma_k \, f^{(k)}(x_0)/k! \cdot (x - x_0)^k$$

The first-order approximation $f(x) \approx f(x_0) + f'(x_0)(x - x_0)$ is the **linearization** of f at x_0.

Partial Derivatives and Gradients

For $f: \mathbb{R}^n \to \mathbb{R}$, the **partial derivative** $\partial f/\partial x_i$ measures the rate of change with respect to x_i while holding all other variables fixed. The **gradient** collects all partial derivatives:

$$\nabla f(x) = [\partial f/\partial x_1, \partial f/\partial x_2, \ldots, \partial f/\partial x_n]^T \in \mathbb{R}^n$$

Direction of Steepest Ascent

The gradient $\nabla f(x)$ points in the direction of greatest increase of f at x. Its negative points toward steepest descent.

Orthogonality to Level Sets

The gradient is always perpendicular to the level curves (contours) of f — a key geometric property.

Gradients of Vector-Valued Functions

Jacobian Matrix

For $f: \mathbb{R}^n \to \mathbb{R}^m$, the **Jacobian** $J \in \mathbb{R}^{m \times n}$ collects all partial derivatives: $J_{ij} = \partial f_i / \partial x_j$

The Jacobian generalizes the gradient to vector-valued functions and is essential for the chain rule in higher dimensions.

Gradients of Matrices

When differentiating with respect to a matrix $X \in \mathbb{R}^{m \times n}$, the gradient df/dX is a matrix of the same size. Key identities:

- $d/dX \, \mathrm{tr}(AX) = A^T$
- $d/dX \, \det(X) = \det(X) \, X^{-T}$
- $d/dX \, x^T A x = (A + A^T)x$

Backpropagation and Automatic Differentiation

Backpropagation is an efficient algorithm for computing gradients in neural networks by applying the chain rule systematically from output to input layers. **Automatic differentiation (AD)** generalizes this to arbitrary computational graphs.

Forward Pass

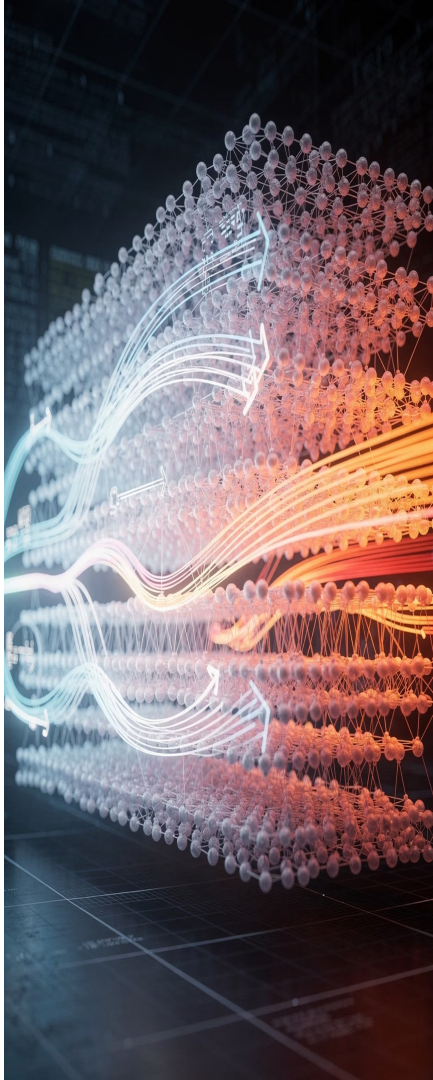
Compute function values layer by layer, storing intermediate results for use in the backward pass.

Backward Pass

Apply the chain rule in reverse: propagate gradients from the loss back through each layer to compute parameter gradients.

Chain Rule

For composed functions $f = g \circ h$: $df/dx = (dg/dh)(dh/dx)$. The Jacobian of the composition is the product of Jacobians.



Higher-Order Derivatives

Hessian Matrix

For $f: \mathbb{R}^n \to \mathbb{R}$, the **Hessian** $H \in \mathbb{R}^{n \times n}$ collects all second-order partial derivatives: $H_{ij} = \partial^2 f / \partial x_i \partial x_j$. The Hessian is symmetric (by Schwarz's theorem) and characterizes the local curvature of f.

Role in Optimization

- Positive definite $H \to$ local minimum
- Negative definite $H \to$ local maximum
- Indefinite $H \to$ saddle point
- Newton's method uses $H^{-1}\nabla f$ for faster convergence than gradient descent

Linearization and Multivariate Taylor Series

The **multivariate Taylor series** approximates a smooth function $f: \mathbb{R}^n \to \mathbb{R}$ near a point x_0:

> $f(x) \approx f(x_0) + (\nabla f(x_0))^T(x-x_0) + \frac{1}{2}(x-x_0)^T H(x_0)(x-x_0) + \dots$

0th Order

$f(x) \approx f(x_0)$ — constant approximation at the expansion point.

1st Order (Linearization)

$f(x) \approx f(x_0) + \nabla f(x_0)^T(x-x_0)$ — tangent hyperplane approximation.

2nd Order

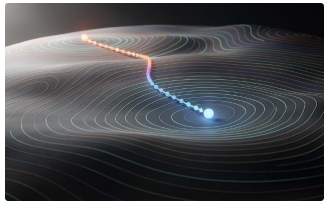
Includes the Hessian term — captures curvature and is used in Newton-type optimization methods.

Useful Gradient Identities

Several gradient identities appear repeatedly in machine learning derivations. Mastering these avoids lengthy re-derivations and accelerates model development.

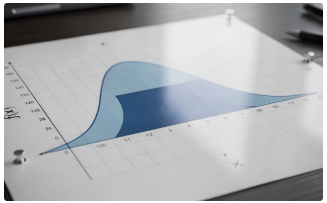
Expression	Gradient
$f(x) = a^T x$	$\nabla f = a$
$f(x) = x^T A x$ (A symmetric)	$\nabla f = 2Ax$
$f(X) = \mathrm{tr}(AX)$	$\nabla f = A^T$
$f(X) = \mathrm{tr}(X^T A X)$	$\nabla f = (A + A^T)X$
$f(X) = \det(X)$	$\nabla f = \det(X)\, X^{-T}$

Vector Calculus in Machine Learning



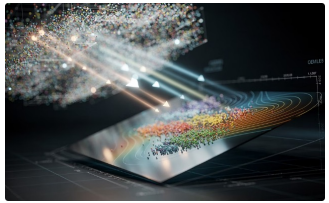
Gradient Descent

Parameters are updated in the direction of the negative gradient: $\theta \leftarrow \theta - \eta \nabla L(\theta)$. The gradient of the loss L with respect to parameters θ is computed via backpropagation.



Maximum Likelihood

Finding MLE parameters requires differentiating the log-likelihood with respect to parameters and setting the gradient to zero — a direct application of vector calculus.



PCA and Regression

Both PCA and linear regression involve minimizing reconstruction/residual errors — solved by setting gradients to zero and using the normal equations derived from vector calculus.

From Theory to Machine Learning

Linear Algebra → Regression

Least-squares solutions via normal equations: $\theta = (A^TA)^{-1}A^Tb$. Matrix operations are the backbone of linear regression.

Geometry → PCA

Orthogonal projections onto lower-dimensional subspaces minimize reconstruction error — the geometric heart of dimensionality reduction.

SVD → Compression

Low-rank matrix approximations via SVD enable efficient data compression, denoising, and latent factor models.

Calculus → Training

Gradients and backpropagation enable optimization of model parameters via gradient descent in all modern ML frameworks.

Symbol Reference: Linear Algebra & Geometry

Symbol	Meaning
$\mathbf{x}, \mathbf{y}, \mathbf{z}$	Vectors (bold lowercase)
$\mathbf{A}, \mathbf{B}, \mathbf{C}$	Matrices (bold uppercase)
$\mathbf{A}^\top$	Transpose of matrix A
$\mathbf{A}^{-1}$	Inverse of matrix A
$\langle \mathbf{x}, \mathbf{y} \rangle$	Inner product of x and y
$\mathrm{rk}(\mathbf{A})$	Rank of matrix A
$\mathrm{Im}(\Phi)$	Image of linear mapping Φ
$\ker(\Phi)$	Kernel (null space) of Φ
$\dim$	Dimensionality of vector space
∇	Gradient operator
$\partial f / \partial x$	Partial derivative of f w.r.t. x

Symbol Reference: Decompositions & Calculus

Symbol	Meaning
λ	Eigenvalue or Lagrange multiplier
E_λ	Eigenspace for eigenvalue λ
$\mathrm{tr}(\mathbf{A})$	Trace of A (sum of diagonal)
$\det(\mathbf{A})$	Determinant of A
σ_i	Singular values (SVD)
U, Σ, V^T	SVD factors of a matrix
df/dx	Total derivative of f w.r.t. x
J	Jacobian matrix
H	Hessian matrix (second derivatives)
$\mathfrak{L}$	Lagrangian
ONB	Orthonormal basis